

COURSE	CECS 378	TERM	Su26
EXAM	Exam 1	POINTS	60

Exam 1 — Introduction to Computer Security

 **PRINT CLEARLY. UNNAMED EXAMS CANNOT BE RETURNED OR GRADED.**

NAME: _____

STUDENT ID#: _____ **DATE:** _____

★ ANSWER KEY — NOT FOR DISTRIBUTION ★

 **Directions.** Write on the exam paper. **Multiple choice:** circle the single best answer. **True/False:** circle the correct option, (a) True or (b) False. **Short answer:** write in the space provided. You may use one 3"×5" card of notes (both sides, handwritten only). **No electronic devices of any kind are allowed on your desk during the exam.**

SECTION 1 — Multiple Choice (4 pts each, 20 pts total)

1. (4 pts) Which of the following best describes a **symmetric-key cipher**?

- (a) The sender encrypts with a public key; the receiver decrypts with a private key.
- (b) **The same secret key is used for both encryption and decryption.**
- (c) No key is required; security relies entirely on algorithm secrecy.
- (d) A digital signature is produced as a side effect of encryption.

Answer: b Symmetric ciphers (e.g., AES) use a single shared secret for both directions. (a) describes asymmetric encryption; (c) is “security through obscurity,” which is not the symmetric model; (d) conflates encryption with signing.

2. (4 pts) A cryptographic hash function is said to be **collision resistant** if:

- (a) Given a hash value, it is computationally infeasible to find any input that produces it.
- (b) The hash output is exactly the same length as the input.
- (c) **It is computationally infeasible to find two distinct inputs that produce the same hash output.**
- (d) The hash function runs in constant time regardless of input length.

Answer: c Collision resistance means no adversary can efficiently find $x \neq y$ s.t. $H(x) = H(y)$. (a) is preimage resistance (a different property); (b) is false — hashes compress variable-length input to fixed-length output; (d) is unrelated to security.

3. (4 pts) In RSA, a message is encrypted for a recipient by using:

- (a) The sender's private key.
- (b) A shared session key negotiated via Diffie-Hellman.
- (c) The recipient's private key.

(d) **The recipient's public key.**

Answer: d RSA encryption uses the *recipient's public key*; only the recipient's matching private key can decrypt. (a) describes signing, not encryption.

4. (4 pts) A classic **stack buffer overflow** typically overwrites which memory region to redirect control flow?

- (a) The heap allocation header.
- (b) The process's environment variables.
- (c) **The saved return address on the stack frame.**
- (d) The kernel's system-call dispatch table.

Answer: c By writing past the end of a stack-allocated buffer, an attacker can overwrite the saved return address (and optionally the saved frame pointer) to redirect execution to attacker-controlled code or a gadget chain.

5. (4 pts) The **principle of least privilege** states that:

- (a) A subject should be granted the maximum privileges needed for any possible future task.
- (b) All users must share a single privileged account to simplify auditing.
- (c) Privileges are determined by a mandatory label assigned at object creation time.
- (d) **A subject should be granted only the minimum privileges necessary to perform its current task.**

Answer: d Least privilege limits the damage a compromised process or user can cause. (a) violates the principle; (b) is the opposite of good practice; (c) describes mandatory access control labels, not least privilege.

SECTION 2 — True / False (2 pts each, 10 pts total)

6. (2 pts) T / F. AES is an example of an asymmetric (public-key) cipher.

- (a) True
- (b) **False**

Answer: False AES is a *symmetric* block cipher — one shared secret key. RSA and ECC are the asymmetric examples.

7. (2 pts) T / F. A cryptographic hash function maps an arbitrary-length input to a fixed-length output.

- (a) **True**
- (b) False

Answer: True Compression to a fixed-size digest (e.g., 256 bits for SHA-256) regardless of input length is a defining property of a hash function.

8. (2 pts) T / F. To send a confidential message with RSA, the sender encrypts it with their own private key.

- (a) True
- (b) **False**

Answer: False Confidentiality uses the *recipient's public key* (only the recipient's private key decrypts). Encrypting with one's *own private key* is signing, not secrecy.

9. (2 pts) T / F. A non-executable stack (NX / W $\oplus$ X) by itself defeats return-oriented programming (ROP) attacks.
- (a) True
(b) **False**

Answer: False ROP chains *existing* executable code (gadgets ending in `ret`), so it needs no injected code and sidesteps NX. ASLR and CFI are the complementary defenses.

10. (2 pts) T / F. The principle of least privilege limits the damage a compromised process can cause.
- (a) **True**
(b) False

Answer: True Granting only the privileges a task currently needs shrinks the blast radius when that process or account is compromised.

SECTION 3 — Short Answer (10 pts each, 30 pts total)

11. (10 pts) Briefly explain **why symmetric ciphers are generally used to encrypt bulk data** in practice, while asymmetric ciphers are typically used only for key exchange or digital signatures. What fundamental trade-off motivates this hybrid design?

Key answer: Symmetric ciphers (AES, ChaCha20) operate on fixed-size blocks or streams using simple arithmetic operations and are therefore orders of magnitude faster than asymmetric algorithms. Asymmetric ciphers (RSA, ECDH) rely on hard mathematical problems (integer factorization, discrete logarithm) whose computations are slow — encrypting megabytes of data with RSA directly is impractical. The hybrid design uses the asymmetric algorithm only once (to securely exchange a symmetric session key), then uses the fast symmetric cipher for all subsequent bulk data. This gives both the key-distribution benefit of public-key cryptography and the performance of symmetric encryption.

Partial credit: award 5–7 pts for identifying the speed difference and that hybrid design exists, even without full mechanistic explanation.

12. (10 pts) Name and briefly describe **two** OS/compiler-level mitigations for stack buffer overflow attacks. For each mitigation, also state one way an attacker might attempt to circumvent it.

Key answer (any two of the following; 5 pts each):

- **Stack canary.** The compiler inserts a random sentinel value (canary) between local buffers and the saved return address. Before a function returns, the canary is checked; a mismatch aborts the process. *Circumvention:* leak the canary value via a format-string or information-disclosure vulnerability, then overwrite it with the correct value when crafting the overflow payload.
- **Non-executable stack (NX / W $\wedge$ X).** Pages are marked either writable or executable, never both. Shellcode injected onto the stack cannot run. *Circumvention:* return-oriented programming (ROP) — chain together existing executable code gadgets (“`ret`” sequences in libraries) to build arbitrary computations without injecting new code.
- **Address Space Layout Randomization (ASLR).** The OS randomizes the base addresses of the stack, heap, and shared libraries at each execution. Hardcoded jump targets in the exploit become unreliable. *Circumvention:* brute-force the address space (feasible on 32-bit);

leak an address from a running process via a format-string or memory-disclosure bug to de-randomize the layout.

13. (10 pts) Compare **Discretionary Access Control (DAC)** and **Mandatory Access Control (MAC)**. For each model: (i) who controls access decisions, and (ii) give one concrete example of a system or OS feature that uses it.

Key answer:

DAC. The *resource owner* (typically the user who created the object) controls access. The owner sets permissions and may grant them to other subjects at will. Example: Unix/Linux file permissions (`chmod`, `chown`) and POSIX ACLs — a user can `chmod 777` their own file to grant global read/write access.

MAC. Access decisions are controlled by the *system policy*, enforced by the OS kernel (or a reference monitor) and cannot be overridden by individual users. Subjects and objects carry labels (e.g., security clearance levels); the policy (Bell-LaPadula, Biba, etc.) determines whether a given subject-to-object access is permitted. Example: SELinux enforces MAC policy on RHEL/Fedora — even root cannot read a file if the SELinux policy forbids it.

Key distinction: in DAC a user can accidentally or maliciously share sensitive data by relaxing permissions; MAC prevents this because the policy is mandatory and users cannot weaken it.

Partial credit: 3–4 pts per model for a correct description without a concrete example.

End of exam. Total: 60 points.